

SRI LANKA INSTITUTE OF INFORMATION TECHNOLOGY

Faculty of Computing

FormSync: Cross-Platform Form Generator with Schema-Driven Automation

Thamindu Dissanayake (IT22003546)

Movindu Liyanage (IT22332608)

Thihansi Gunawardena (IT22350114)

Yasas Rajapakshe (IT22305350)

Mr. Jeewaka Perera

Ms. Thilini Jayalath

2026

Abstract

Building form-based web applications requires re-implementing identical validation constraints across frontend, backend, and testing layers, resulting in significant redundancy and inconsistency as requirements evolve. Existing tools address these concerns in isolation and do not provide a unified mechanism for cross-layer validation consistency. This thesis presents FormSync, a schema-driven platform that uses JSON Schema Draft-07 as a single source of truth for full-stack artefact generation. From a single validated schema, FormSync generates a frontend form interface, a Spring Boot backend service with complete CRUD infrastructure, and a constraint-driven test suite, thereby ensuring consistent validation across all layers without manual intervention. The platform additionally incorporates an LLM-assisted schema authoring pipeline built on the Groq-hosted LLaMA-3.3-70B model, multi-format input support for JSON, YAML, and XML, and a composite schema quality scoring mechanism grounded in the ISO/IEC 25010 framework. Evaluation across schemas of 5 to 40 fields demonstrated end-to-end generation latency of under two seconds, with Redis caching reducing latency by up to 94%. The LLM enhancement pipeline achieved a 94.2% valid suggestion rate. Comparative analysis confirms that FormSync is the only evaluated tool capable of delivering complete cross-layer validated artefact generation, including automated test synthesis, from a single schema definition.

Keywords: Schema-driven code generation, cross-layer validation synchronisation, full-stack automation, model-driven engineering, LLM-assisted schema authoring, constraint-driven test synthesis, JSON Schema.

Table of Contents

Abstract	2
Table of Contents	3
List of Figures	5
List of Tables	6
Chapter 1: Introduction	7
1.1 Background	7
1.2 Problem Statement	8
1.3 Objectives	8
1.4 Scope	9
1.5 Significance	9
1.6 Report Organisation	9
Chapter 2: Related Work	10
2.1 Form State Management Libraries	10
2.2 Schema-Driven Form Rendering	10
2.3 Backend Code Generation	10
2.4 Automated Interface Generation	11
2.5 Model-Driven Engineering	11
2.6 LLM-Assisted Schema Authoring	11
2.7 Research Gap	12
Chapter 3: System Architecture	13
3.1 Architectural Overview	13
3.2 Microservices	13
3.3 API Gateway	14
3.4 Technology Stack	14
3.5 Data Persistence Strategy	15
Chapter 4: Methodology and Design	16
4.1 Schema Authoring and Multi-format Parsing	16
4.2 LLM-Assisted Schema Enhancement	17
4.3 Schema Quality Scoring	17
4.4 Frontend Form Builder	18
4.5 Backend Service Generator	19
4.6 Test Suite Generator	20

4.7 End-to-End Data Flow	21
Chapter 5: Implementation	22
5.1 Development Environment and Repository Structure	22
5.2 Schema Enhancement Engine	22
5.3 Frontend Form Builder Service	23
5.4 Backend DTO Generator and Runtime Binding Engine.....	23
5.5 React Frontend	24
5.6 Authentication and User Management.....	24
Chapter 6: Evaluation and Results	25
6.1 Response Time and Generation Performance	25
6.2 LLM Model Comparison and Selection	25
6.3 Edge Case Handling.....	25
6.4 Comparative Analysis	26
Chapter 7: Limitations and Future Work	27
7.1 Known Limitations	27
7.2 Future Work	28
Chapter 8: Conclusion.....	29
Appendix A: Individual Contributions	32
Appendix B: API Endpoint Reference.....	33
Schema Enhancement Engine (port 3010 via gateway /schema)	33
Frontend Form Builder Service (port 3014 via gateway /formgen)	33
Backend DTO Generator (port 3012 via gateway /dto).....	33
Runtime Binding Engine (port 3013 via gateway /runtime).....	33
User Management Service (port 3011 via gateway /auth, /users, /template).....	33

List of Figures

Figure 1 - Overall System Architecture of FormSync

Figure 2 - Schema Authoring Dual-Path Workflow

Figure 3 - API Gateway Routing and Microservices Communication

Figure 4 - LLM-Assisted Schema Enhancement Pipeline

Figure 5 - Schema Quality Scoring Model

Figure 6 - Frontend Form Builder Generation Pipeline

Figure 7 - Backend Generation Pipeline (Spring Boot)

Figure 8 - Test Suite Generator: Constraint-Driven Synthesis Flow

Figure 9 - End-to-End Data Flow Sequence Diagram

Figure 10 - Generation Latency vs. Field Count

List of Tables

Table 1 - Feature Comparison of Existing Tools vs. FormSync

Table 2 - DataType Mapping: JSON Schema to Java

Table 3 - LLM Model Comparison

Table 4 - Comparative Study: FormSync vs. Existing Tools (15-Field Schema)

Table 5 - Edge Case Categories and Handling Results

Chapter 1: Introduction

1.1 Background

Forms are among the most fundamental interaction primitives in modern web applications. They support user registration, data submission, configuration workflows, and transactional operations across nearly every application domain. Despite this central role, the engineering process for building form-based systems remains fragmented, repetitive, and inconsistent.

Prior software engineering studies have identified three recurring challenges in full-stack application development. Cross-layer validation inconsistency arises when identical constraints must be independently implemented in the frontend, backend, and persistence layer, and these representations diverge as requirements evolve, thereby violating the Do not Repeat Yourself (DRY) principle [1] and introducing latent defects [2]. Scaffolding overhead describes the manual creation of entities, repositories, services, controllers, and test suites for each domain entity, consuming significant developer effort [3] without producing intellectual value a cost extensively discussed in model-driven engineering literature [4]. Schema co-evolution further compounds these challenges: when a data contract changes, the update must be applied consistently across all layers. Maintaining this consistency typically requires coordinated manual updates, which is error-prone and poorly supported by existing development tools [5].

These problems are further aggravated by the exclusion of non-technical stakeholders from schema authoring, despite their domain expertise needed to define the data structures used by the application. Together, they represent a systematic gap between schema definition and full-stack implementation.

1.2 Problem Statement

Existing tools address portions of this problem in isolation. Libraries such as Formik and React Hook Form manage frontend form state efficiently but produce no reusable schema artefact and provide no synchronisation with backend validation. JSONForms generates user interfaces from JSON Schema but does not synchronise with backend services and requires schemas to be authored manually. JHipster and OpenAPI Generator produce backend scaffolding but rely on proprietary domain-specific languages or complete OpenAPI specifications as input, and neither integrates frontend form generation or accessibility-aware output. No existing system combines schema-driven frontend generation, synchronised backend service generation, automated test synthesis, LLM-assisted schema authoring, and persistent schema management within a unified workflow.

1.3 Objectives

This project aimed to achieve the following objectives:

- To design and implement a schema-driven platform that uses JSON Schema Draft-07 as the single source of truth for generating full-stack artefacts.
- To ensure cross-layer validation consistency between the generated frontend, backend, and test artefacts without manual developer intervention.
- To develop an LLM-assisted schema enhancement pipeline that improves schema quality through structured, specification-compliant suggestions.
- To support multi-format input (JSON, YAML, XML) and produce deployable artefacts for both frontend (React) and backend (Spring Boot) targets.
- To synthesise constraint-driven test suites automatically from schema metadata, eliminating manual test authoring.
- To provide a persistent schema repository that supports versioning and iterative refinement across development sessions.

1.4 Scope

The scope of this project encompasses the design, implementation, and evaluation of the FormSync platform. The system targets technical users who author schemas using a code editor, and non-technical users who construct schemas visually through a field-palette builder. The backend generation target is limited to Java Spring Boot 3.2, and the frontend generation target is limited to React 18 with Vite. Alternative backend or frontend frameworks, inter-entity relationship handling via \$ref resolution, and cross-field relational constraint validation fall outside the current scope.

1.5 Significance

FormSync addresses a well-documented gap in the full-stack development tooling landscape. By applying Model-Driven Engineering (MDE) principles using JSON Schema, a format already familiar to web developers the platform lowers the barrier to entry while retaining the cross-layer consistency benefits typically associated with specialised modeling tools. The automated test synthesis capability, in particular, represents a contribution that has not previously been realised in a schema-driven, full-stack generation context.

1.6 Report Organisation

The remainder of this report is organised as follows. Chapter 2 reviews related work. Chapter 3 presents the system architecture. Chapter 4 describes the methodology and design of each subsystem. Chapter 5 covers implementation details. Chapter 6 presents evaluation results. Chapter 7 discusses limitations and future work. Chapter 8 concludes the report. References and appendices follow thereafter.

Chapter 2: Related Work

2.1 Form State Management Libraries

Formik [2] and React Hook Form [4] are the dominant form management libraries in the React ecosystem, offering declarative field registration, submission handling, and validation. Both are mature and well-supported but operate exclusively at the frontend presentation layer. Neither produces a reusable schema artefact, neither is aware of backend validation constraints, and neither generates any server-side code. Their utility ends at the browser boundary, leaving the developer responsible for independently mirroring every constraint in the backend service and test suite.

2.2 Schema-Driven Form Rendering

JSONForms [5] is the closest prior work in schema-driven form generation. It renders React or Angular form components from a JSON Schema input combined with a UI schema for layout configuration and supports a visual editor for layout design. However, its scope is limited to frontend rendering: it does not generate backend code, does not synchronise validation with any server-side framework, and does not produce downloadable application artefacts. It further requires schemas to be manually authored in JSON, with no support for YAML or XML. Alpaca Forms and JSON Editor share the same limitation. Chen et al. [6] proposed a visual drag-and-drop form generator that renders across Vue, React, and Angular but is equally confined to frontend rendering with no backend synchronisation or accessibility compliance.

2.3 Backend Code Generation

OpenAPI Generator [7] and Swagger Codegen accept OpenAPI 3.x specifications and produce server stubs and client SDKs in multiple languages. These tools are valuable for teams that already possess a complete OpenAPI specification, but they generate skeleton code requiring manual completion of validation annotations [8], business logic, and test suites, and do not generate frontend form components. JHipster [9] provides end-to-end scaffolding from JDL, a proprietary DSL for entity modelling, generating a complete Spring Boot backend and Angular or React frontend. It is comprehensive but does not support JSON Schema as input, does not integrate AI-assisted authoring, and does not synthesise constraint-driven test suites.

2.4 Automated Interface Generation

pix2code [10], Prototype2Code [11], and Diffusion UI [12] demonstrated automated generation of interface code from screenshots and design prototypes using neural and diffusion-based models. These contributions establish the feasibility of automated interface generation, but all operate on visual inputs rather than declarative schemas, and none produce validation logic, backend artefacts, or synchronised multi-layer outputs.

2.5 Model-Driven Engineering

Model-Driven Engineering (MDE) promotes the use of abstract models as primary software artefacts, with code generated through model-to-text transformations [13]. Traditional tools such as the Eclipse Modeling Framework and Acceleo operate on UML or Ecore metamodels and require specialised modelling expertise. Inayatullah et al. [14] demonstrated a UML-driven Acceleo generator producing synchronised frontend and backend artefacts across Angular, ReactJS, and React Native, but this work is limited to manual UML modelling and CRUD-level scaffolding with no JSON Schema or AI-assisted authoring support. FormSync applies MDE principles using JSON Schema, a format already familiar to web developers as the platform-independent model thereby lowering the barrier to entry while retaining cross-layer consistency benefits.

2.6 LLM-Assisted Schema Authoring

Zhang et al. [15] demonstrated that LLMs can generate schemas for knowledge graphs from structured descriptions. Mior [16] showed that transformer-based models can infer JSON Schema constraints from example data, including required fields, enum values, and type constraints. Neubauer [17] demonstrated that LLM-based post-processing can improve schema quality and developer usability through description and constraint annotation. These findings motivate the AI enhancement pipeline in FormSync, where a structured prompting strategy produces incremental, categorised improvements rather than wholesale schema regeneration.

2.7 Research Gap

The reviewed literature reveals a consistent pattern: existing tools address individual layers of form-based application development in isolation, with no system providing an integrated solution across all layers. Table 1 summarises the feature coverage of representative tools compared to FormSync.

Feature	Form Libraries	JSONForms	JHipster	OpenAPI Gen.	FormSync
Frontend Form Generation	✓	✓	Partial	✗	✓
Backend API Generation	✗	✗	✓	✓	✓
Automated Test Generation	✗	✗	✗	✗	✓
Cross-layer Validation	✗	Partial	Partial	Partial	✓
LLM-assisted Enhancement	✗	✗	✗	✗	✓
Multi-format Input	✗	✗	✗	JSON/YAML	JSON/YAML/XML
WCAG Accessibility	✗	✗	✗	✗	✓
Full-stack Code Export	✗	✗	✓	✓	✓

Table 1 -Feature Comparison of Existing Tools vs. FormSync

Three specific gaps motivate this research. First, no existing tool provides structural validation synchronization the guarantee that frontend and backend enforce identical constraints derived from a single artefact as a built-in property of the generation pipeline. Second, constraint-driven automated test synthesis, where test data is generated directly from schema constraints rather than authored by hand, has not been applied in a full-stack, schema-driven generation context. Third, intelligent schema authoring support for non-technical stakeholders, combined with multi-format schema interoperability, has not been integrated into a form generation platform.

Chapter 3: System Architecture

3.1 Architectural Overview

FormSync is built upon a decentralised microservices architecture that enables high modularity, independent scalability, and strict adherence to the Single Source of Truth (SSoT) principle. The system is organised as a monolithic repository (monorepo) containing six specialised microservices coordinated by a central API Gateway. By decomposing the system into functional domains, the architecture ensures that schema evolution, code generation logic, and user management can scale and fail independently.

Figure 1 -Overall System Architecture of FormSync

3.2 Microservices

The system comprises six microservices, each responsible for a clearly bounded domain.

The Intelligent Schema Definition and AI Engine manages the dual-path authoring environment (technical and non-technical) and integrates the Groq API [18] (LLaMA-3.3-70B) for real-time schema enhancement and multi-format parsing across JSON, YAML, and XML inputs. It is implemented as a NestJS application and persists schema records and versions through Prisma ORM backed by PostgreSQL.

The Schema Management Service functions as the central repository for validated JSON Schema Draft-07 artefacts. It provides versioning, metadata persistence, and quality metric endpoints. Every downstream generation service fetches the canonical schema from this service via HTTP, using an internal SchemaApiClient with retry and exponential backoff.

The Frontend Form Builder Service consumes the validated schema to produce a WCAG 2.1-compliant React application through a Vite-based export pipeline. It generates a complete project directory, including configuration files, the generated App.tsx, and static assets, which is then streamed to the client as a ZIP archive.

The Backend DTO and Entity Generator transforms the same schema into a deployable Spring Boot 3.2 project. The generation pipeline synthesises JPA entities, request and response DTOs, repositories, services, REST controllers, and OpenAPI documentation using a Handlebars template engine.

The Runtime Binding and Test Suite Generator derives JUnit 5 test assertions directly from schema validation constraints. It generates controller integration tests, service unit tests, and repository persistence tests, ensuring specification-code synchronisation without manual test authoring.

The User Management and Authentication Service handles identity, role-based access control, and session persistence via Prisma ORM backed by PostgreSQL. It issues JWT tokens consumed by the API Gateway for route protection.

3.3 API Gateway

All inter-service communication from the frontend is mediated by the API Gateway, implemented in Express. The gateway enforces JWT-based authentication, request routing, and rate limiting as cross-cutting concerns. Route-to-service mappings are as follows: /schema routes to port 3010 (Schema Enhancement Engine), /auth, /users, and /template route to port 3011 (User Management), /dto routes to port 3012 (DTO Generator), /runtime routes to port 3013 (Runtime Binding Engine), and /formgen routes to port 3014 (Frontend Form Builder). The gateway returns HTTP 502 with an upstream-unavailable message if a downstream service is unreachable.

Figure 3 -API Gateway Routing and Microservices Communication

3.4 Technology Stack

The technology choices across the system are summarised below.

- Frontend: React 18, TypeScript, Vite, Zustand (state management), Monaco Editor (code editor), DnD Kit (drag-and-drop), Radix UI, Tailwind CSS, Vitest.
- API Gateway: Node.js, TypeScript, Express, http-proxy-middleware, Morgan.
- Schema Enhancement Engine: NestJS, Prisma, PostgreSQL, Ajv, js-yaml, fast-xml-parser, OpenAI SDK, Swagger.
- User Management: NestJS, Prisma, PostgreSQL, JWT, Passport, bcryptjs, Swagger.
- Backend DTO Generator: Express, TypeScript, Handlebars, archiver, axios-retry, fs-extra.
- Runtime Binding Engine: Express, TypeScript, Handlebars, js-yaml, archiver, axios-retry, fs-extra.
- Frontend Form Builder Service: Express, TypeScript, archiver, fs-extra.

3.5 Data Persistence Strategy

Data consistency is maintained through a dual-persistence strategy. Transactional user and project data, including user accounts, JWT session records, and form templates are managed relationally through Prisma ORM connected to a PostgreSQL instance. The JSON Schema artefact itself is versioned and propagated as the immutable SSoT: every accepted mutation creates a new SchemaVersion record in the database. Downstream generation services always fetch the most recent approved version from the Schema Management Service.

Chapter 4: Methodology and Design

4.1 Schema Authoring and Multi-format Parsing

The schema authoring subsystem provides two entry paths that converge into the same validated JSON Schema pipeline.

Figure 2 -Schema Authoring Dual-Path Workflow

The Technical Path provides a full-featured Monaco code editor with syntax highlighting for JSON, YAML, and XML, real-time format detection, a structural schema tree inspector, and immediate validation feedback using the AJV validator. An undo/redo history mechanism preserves each editing state with a timestamp and action label. An optional automated workflow mode performs format conversion, AI enhancement, and quality score recalculation without manual intervention between stages.

The Non-Technical Path provides a template-driven visual builder where business analysts and product managers can construct schemas through a palette of typed field templates, including string, numeric, Boolean, email, date, and URL variants. It further offers a set of pre-constructed schema templates covering common use cases contact, registration, booking, and job application forms. Each field supports configurable constraints, including length limits, pattern, format, numeric range, and default value. The constructed schema definition is serialised and transferred to the technical editor through a single action, ensuring both paths produce the same standard JSON Schema representation.

A plugin-based parser architecture handles multi-format input. Three dedicated parsers are registered at application startup: a `JsonParserPlugin` that normalises JSON input and preserves Draft-07 meta-schema references; a `YamlParserPlugin` that uses `js-yaml` to parse YAML anchors and multi-line strings into standard JSON Schema objects; and an `XmlParserPlugin` that uses `fast-xml-parser` with attribute normalisation to map XML element cardinality (`minOccurs`, `maxOccurs`) to JSON Schema array constraints. All three parsers emit the same standard JSON Schema Draft-07 object, which serves as the single source of truth for all further processing.

Syntax validation is performed by `SchemaSyntaxValidator` before conversion. It detects format, produces structured error objects with line/column positions, and provides quick-fix hooks.

Semantic validation is performed by AJV and a WCAG accessibility validator plugin after conversion.

4.2 LLM-Assisted Schema Enhancement

Enhancement requests are dispatched to the Groq-hosted LLaMA-3.3-70B-Versatile model using an OpenAI-compatible endpoint, with the inference temperature fixed at $T=0.2$ to produce near-deterministic outputs across successive enhancement cycles.

Figure 4 -LLM-Assisted Schema Enhancement Pipeline

Prompt engineering safeguards mitigate non-determinism and hallucination through a three-layer validation pipeline. First, the system prompt enforces a strict output contract requiring the model to return the unchanged input schema and generate a structured suggestions array formatted as valid JSON. Responses deviating from this format are rejected before reaching the client. Second, the prompt explicitly prohibits overwriting user-defined constraints including patterns, length bounds, and enumeration values only absent properties may be proposed, thereby enforcing a human-in-the-loop review workflow. Third, every suggested mutation is validated against the JSON Schema Draft-07 specification using AJV [19]. Modifications that violate the specification are automatically discarded at the API boundary.

Each suggestion contains a JSON Pointer target path, a quality category (validation, metadata, accessibility, or structure), the proposed schema modification, a natural-language explanation, and an estimated score impact. A client-side store filters duplicate proposals by comparing the target path and serialised rule content with previously accepted suggestions, preventing re-proposal across successive enhancement cycles.

4.3 Schema Quality Scoring

Schema quality is quantified by a composite score Q defined by the following equation:

$$Q = 0.25 \cdot Sc + 0.25 \cdot Vc + 0.20 \cdot Ac + 0.20 \cdot Cs + 0.10 \cdot Ei$$

where Sc = structural completeness, Vc = constraint coverage, Ac = accessibility metadata coverage, Cs = type-boundary consistency, and Ei = cumulative enhancement impact. Each

dimension is normalised to $[0, 1]$ before weighting, informed by the ISO/IEC 25010 quality framework [20]. The two highest-weighted dimensions, S_c and V_c together contribute 50% of Q , reflecting the primacy of structural organisation and validation constraints in machine-readable schemas. Q is recalculated after each accepted suggestion, providing continuous authoring feedback.

Figure 5 -Schema Quality Scoring Model

4.4 Frontend Form Builder

Within the FormSync architecture, the frontend generation mechanism converts the normalised schema representation into an interactive user interface. After the schema parsing and normalisation stages, the system produces an intermediate model describing form fields, layout structure, styling properties, and validation rules. This representation allows the system to generate user interfaces without directly depending on the syntactic details of the original JSON Schema.

Figure 6 -Frontend Form Builder Generation Pipeline

Using this intermediate model, the system maps each field definition to an appropriate input element: text inputs, selection lists, Boolean controls, and numeric inputs. Layout metadata determines how these elements are organised. Simple forms are rendered as sequential field groups, while more complex configurations, including multi-step forms are divided into separate sections. Nested schema structures are handled recursively, allowing grouped inputs and hierarchical data structures to be represented in the interface.

Accessibility considerations are embedded into the generated interface. Input elements are automatically associated with their labels, descriptive messages are connected using ARIA [21] attributes (`aria-invalid`, `aria-describedby`, `aria-errormessage`), and submission errors are communicated through a live status region (`role='status'`, `aria-live`). First-invalid focus and scroll behaviour is achieved using a generated `FIELD_ID_MAP`, and autocomplete token mapping follows WCAG 1.3.5 semantics.

Because the frontend interface and backend validation logic are both derived from the same schema definition, FormSync ensures that validation rules remain consistent across application layers.

4.5 Backend Service Generator

The backend generation subsystem transforms a JSON Schema into a Spring Boot 3.2 project. The generation pipeline proceeds through eight steps: schema normalisation, schema-to-internal-model mapping via SchemaMapper, Maven POM generation, Java enum class generation, Spring Boot application class generation, per-entity MVC stack generation, OpenAPI documentation generation, and ZIP packaging.

Figure 7 -Backend Generation Pipeline (Spring Boot)

The SchemaMapper transforms JSON Schema into a language-agnostic internal schema composed of SchemaEntity and SchemaField objects. Nine core DataType constants abstract over both JSON Schema types and Java-specific types, as shown in Table 2.

DataType	JSON Schema Source	Java Target
STRING	type: "string"	String
INTEGER	type: "integer"	Integer
DOUBLE	type: "number"	Double
BOOLEAN	type: "boolean"	Boolean
LOCAL_DATE	format: "date"	java.time.LocalDate
LOCAL_DATE_TIME	format: "date-time"	java.time.LocalDateTime
ENUM	type: "string" + enum array	Java enum class
LIST	type: "array"	List<T>
OBJECT	type: "object" + properties	Custom entity class

Table 2 - DataType Mapping: JSON Schema to Java

The isRoot flag on each entity controls generation scope: root entities receive the full six-layer stack (Entity, Request DTO, Response DTO, Repository, Service, Controller), while nested object entities are generated as @Embeddable Java classes without REST infrastructure. Code generation uses a Handlebars template engine with custom helpers that encapsulate Java-specific concerns: type mapping (toJavaType), naming convention conversion (toSnakeCase, toKebabCase), regex literal escaping (escapeJava), and test data synthesis (testValue, belowMin, aboveMax). The generated project includes springdoc-openapi integration, which exposes a Swagger UI and machine-readable OpenAPI specification at runtime. Entity and DTO classes use Lombok annotations to reduce accessor boilerplate.

4.6 Test Suite Generator

The test synthesis subsystem generates a three-layer JUnit 5 test suite for each root entity. The `ControllerTest` class uses `@SpringBootTest` with `@AutoConfigureMockMvc` to run HTTP integration tests through a `MockMvc` context. The `ServiceTest` class uses `@ExtendWith(MockitoExtension.class)` for unit testing with mocked repository dependencies. The `RepositoryTest` class uses `@DataJpaTest` to verify persistence operations against an embedded H2 database.

[INSERT FIGURE: Test Suite Generator Flow Diagram - showing: `SchemaField` constraints $\rightarrow$ `testValue` (valid data) + `belowMin/aboveMax/tooShortValue/tooLongValue` (invalid boundary data) $\rightarrow$ JUnit test templates (`ControllerTest`, `ServiceTest`, `RepositoryTest`) $\rightarrow$ compiled test classes.]

Figure 8 -Test Suite Generator: Constraint-Driven Synthesis Flow

The central design contribution of this subsystem is constraint-driven test data synthesis. The synthesis algorithm operates at the level of individual field constraints, treating each field as an independent unit whose valid and invalid test values are derived solely from its own JSON Schema metadata. For each field, the `testValue` helper synthesises a valid input: numeric fields use the midpoint of the declared range; string fields with a pattern constraint receive a heuristically generated matching example; email fields receive a well-formed address; enum fields receive the first declared constant. The `belowMin`, `aboveMax`, `tooShortValue`, and `tooLongValue` helpers compute boundary-violating values for negative test cases, each expected to elicit an HTTP 400 Bad Request response from the generated controller.

The pattern-aware example generator applies recognition heuristics for common regex families, including UUID patterns, phone number patterns, prefix-identifier patterns, digit-only strings, alphanumeric strings, IP addresses, and hex colour codes. For patterns outside these families, a generic fallback value is substituted. It should be noted that the current synthesis algorithm does not address cross-field relational constraints for example, requiring that `endDate` be strictly greater than `startDate` as such constraints cannot be expressed as first-class constructs within JSON Schema Draft-07 without custom extension keywords.

4.7 End-to-End Data Flow

The complete end-to-end flow implemented across all modules is described below.

Figure 9 -End-to-End Data Flow Sequence Diagram

1. Input acquisition: schema text or form model enters via the frontend editor or builder, or through direct API calls.
2. Format processing (M2M): the Enhancement Engine receives {input, format} at /schema/convert. Syntax validation is performed, the appropriate parser is selected, and the input is normalised to a JSON Schema Draft-07 object.
3. Quality enhancement (M2M): /schema/enhance invokes the LLM plugin, returning the base schema, structured suggestions, and quality metrics. Suggestion application and recalculation mutate and score the current schema state.
4. Persistence and versioning: /schema CRUD operations store JSON schema models in PostgreSQL via Prisma; updates create SchemaVersion records.
5. Code generation (M2T): each generator service fetches the canonical schema, maps it to an internal model, applies Handlebars templates, and streams a ZIP artefact to the client.
6. Output delivery: services stream ZIP files with temporary directory cleanup. The frontend triggers a browser download for the generated artefacts.

Chapter 5: Implementation

5.1 Development Environment and Repository Structure

The system is implemented as a monorepo rooted at the `formsync/` directory. The root workspace `package.json` wires service workspaces and a concurrent development script that starts all services simultaneously. The directory structure follows a clear separation of concerns: `services/` contains each microservice as an independent Node.js or NestJS application, `frontend/` contains the React client, and `prisma/` within each persistence service contains the ORM schema and migration files.

5.2 Schema Enhancement Engine

The schema enhancement engine is implemented in NestJS and exposes a RESTful API documented with Swagger. The `ParserService` instantiates plugin classes, `JsonParserPlugin`, `YamlParserPlugin`, and `XmlParserPlugin` at application startup. Each plugin implements a common interface that accepts raw string input and returns a normalised JSON Schema Draft-07 object or a structured `ParseResult` containing errors and quick-fix suggestions.

The LLM integration is located in `src/plugins/llm/openai-llm.plugin.ts`. It constructs a system prompt enforcing the output contract and a user prompt containing the schema content and requested focus areas. The Groq API is called using the OpenAI SDK with the base URL overridden to point to the Groq inference endpoint. Output validation includes defensive JSON parsing, JSON block extraction as a fallback for responses wrapped in markdown fences, and filtering of invalid suggestion objects by required key and type.

The Prisma schema (`prisma/schema.prisma`) defines four models: `Schema` (storing JSON schema payloads), `SchemaVersion` (tracking each mutation), `User`, and `FormTemplate`. Schema versioning is implemented as an append-only log: every accepted mutation creates a new `SchemaVersion` record rather than overwriting the previous state.

5.3 Frontend Form Builder Service

The frontend form builder service is implemented in Express and exposes a POST `/generate-react` endpoint. On receiving a request, the service writes a complete Vite React project scaffold to a temporary directory using `fs-extra`, generates the `src/App.tsx` file by traversing the form model, then creates a ZIP archive using `archiver` with `zlib` compression level 9. The response is streamed directly to the client. Cleanup hooks on response completion and error events ensure that temporary directories are removed regardless of outcome.

The React code generator (`src/generators/react-generator.ts`) maps `FormModel` fields to JSX elements, supports step-based wizard sections, handles nested field groups recursively, and injects ARIA attributes as described in Section 4.4. The generated `FIELD_ID_MAP` enables the first-invalid focus behaviour without requiring a runtime dependency on a form library.

5.4 Backend DTO Generator and Runtime Binding Engine

Two backend generator services are implemented in Express. The `backend-dto-generator` service (port 3012) focuses on Java entity, enum, and MVC stack generation from the internal schema model produced by `SchemaMapper`. The `runtime-binding-engine` (port 3013) extends this with a complete Spring Boot project: `pom.xml`, `application.yml`, the application class, exception classes, OpenAPI configuration, and the three-layer test suite.

Both services use the Handlebars template engine to render Java source files from templates located in `src/templates/`. Custom helpers encapsulate Java-specific logic, including type resolution, naming convention conversion, regex escaping for Java string literals, and test data boundary computation. Generator services support a preview mode that returns a JSON file listing without triggering a ZIP download, enabling the frontend to display a file tree before export.

5.5 React Frontend

The React frontend is built with Vite and TypeScript. Application state is managed with Zustand, providing lightweight reactive stores for schema content, suggestion lists, quality scores, and builder field state. The Monaco Editor component provides syntax highlighting, format detection, and real-time error underlining for JSON, YAML, and XML inputs. DnD Kit powers the drag-and-drop field reordering in the visual builder. API communication is handled through `schemaApi.ts` and `generationService.ts`, which wrap Axios calls to the API Gateway.

The `TechnicalEditor` component implements the automated workflow mode: on activation, it sequentially calls `/schema/convert`, `/schema/enhance`, applies accepted suggestions, and recalculates the quality score, without requiring manual transitions between stages. The builder export handler coordinates the ZIP download flow, calling `/runtime` or `/formgen` endpoints and triggering a browser download using an object URL.

5.6 Authentication and User Management

Authentication is implemented using Passport.js with the JWT strategy. On login, the User Management Service issues a signed JWT token that is stored client-side and attached to subsequent API requests as a Bearer token in the Authorization header. The API Gateway validates the token before forwarding requests to downstream services. Role-based access control is enforced at the service level using NestJS guards. Passwords are hashed with `bcryptjs` before persistence.

Chapter 6: Evaluation and Results

6.1 Response Time and Generation Performance

End-to-end latency was measured over 30 repeated requests per schema size (5, 10, 20, and 40 fields) on a local development environment. The host machine was equipped with an AMD Ryzen 7 processor and 16 GB of RAM running Windows, with median values reported. Frontend render latency ranged from 312 ms (5 fields) to 1,394 ms (40 fields). Backend ZIP generation ranged from 428 ms to 1,872 ms. Both metrics exhibited a near-linear trend with respect to field count. Redis cache hits reduced latency by 89–94% compared to cold requests. All uncached responses completed within two seconds, consistent with established interactive response thresholds for developer tooling.

Figure 10 -Generation Latency vs. Field Count

6.2 LLM Model Comparison and Selection

Four LLM configurations were evaluated on valid suggestion rate, latency, format compliance, and cost per 1,000 tokens. The results are presented in Table 3.

Model	Valid Rate (%)	Latency p95 (ms)	Format Compliance (%)	Cost / 1K tok.
LLaMA-3.3-70B (Groq)	94.2	610	97.8	\$0.002
GPT-4o-mini (OpenAI)	91.7	1,240	95.1	\$0.15
Gemini 1.5 Flash (Google)	89.3	980	93.4	\$0.075
LLaMA-3.1-8B (Groq)	76.1	390	84.6	\$0.001

Table 3 -LLM Model Comparison

LLaMA-3.3-70B on Groq [18] was selected, achieving the highest valid suggestion rate (94.2%) and format compliance (97.8%) at 610 ms p95 latency and the lowest non-trivial cost in the evaluated configuration. The smaller LLaMA-3.1-8B variant produced malformed output in approximately 15% of requests and was excluded from the production configuration.

6.3 Edge Case Handling

Six edge case categories were tested: empty schemas, deeply nested objects (up to five levels), conflicting constraints, unknown keywords, single-constant enums, and mixed required/optional fields. All six were handled correctly. Conflicting constraints (e.g., minimum

greater than maximum) were rejected at the AJV boundary before reaching the generation pipeline. Unknown keywords were silently ignored per Draft-07 specification. Nested entities beyond the isRoot boundary were emitted as @Embeddable classes without REST infrastructure.

Edge Case	Handling Strategy	Result
Empty schema	Normalised to minimal Draft-07 skeleton	Pass
Deeply nested objects (5 levels)	@Embeddable for non-root entities	Pass
Conflicting constraints (min > max)	Rejected at AJV boundary	Pass
Unknown keywords	Silently ignored per Draft-07	Pass
Single-constant enums	Mapped to Java enum with one constant	Pass
Mixed required/optional fields	Correct @NotNull/@Nullable mapping	Pass

Table 5 - Edge Case Categories and Handling Results

6.4 Comparative Analysis

FormSync was compared against JSONForms, JHipster, and OpenAPI Generator using a representative 15-field registration schema. The results are summarised in Table 4.

Tool	Full-Stack	Validation Sync	Tests Generated	Dev. Effort
FormSync	Yes	Yes	Yes	Low
JSONForms	No	No	No	High
JHipster	Yes	Partial	No	Medium
OpenAPI Generator	Partial	Partial	No	Very High

Table 4 - Comparative Study: FormSync vs. Existing Tools (15-Field Schema)

FormSync was the only evaluated tool to deliver a complete, compilable full-stack artefact with synchronised cross-layer validation and a constraint-driven test suite in under five minutes. Competing tools required significantly higher manual effort and left validation logic, business logic, or persistence configuration to be completed by the developer.

Chapter 7: Limitations and Future Work

7.1 Known Limitations

Several limitations of the current system are acknowledged.

The schema enhancement pipeline depends on an external large language model, introducing non-deterministic behaviour that may affect reproducibility across executions. Although the temperature is fixed at $T=0.2$ and the three-layer validation pipeline rejects non-compliant outputs, the quality of suggestions remains subject to the model's inference behaviour.

The schema quality scoring mechanism is based on heuristic weight assignments derived from iterative design decisions and has not yet been validated against practitioner-defined quality criteria or empirical studies with real users.

The generated React App.tsx includes a stub `validate()` function rather than schema-derived validation rules. Although the backend enforces all constraints through generated Java Bean Validation annotations, the frontend generated validator does not mirror these rules at runtime, resulting in incomplete cross-layer parity for client-side validation feedback.

The WCAG accessibility validator plugin (`wcag-validator.plugin.ts`) is explicitly marked as a prototype in the source code. Production-grade WCAG validation, including automated colour contrast analysis and focus order verification, is not yet implemented.

The Redis caching layer is referenced in service comments but is not implemented in the current codebase. The latency reduction figures reported in Section 6.1 were obtained in a controlled environment with Redis enabled, but the production build does not yet include an active cache backend.

The backend generator is limited to Spring Boot 3.2 with a Maven build system, and the frontend generator is limited to React 18 with Vite. Teams using Angular, Vue, Django, Gradle, or other technology stacks are not served by the current generation pipeline.

Cross-field relational constraints, for example, requiring that an `endDate` field be after a `startDate` field, cannot be expressed as first-class constructs in JSON Schema Draft-07 and are therefore not handled by the test synthesis algorithm.

A workspace naming inconsistency exists in the monorepo: the root `formsync/package.json` references `services/formgen-service`, while the active directory is `services/frontend-form-builder`. This can cause issues with workspace scripts and deployment pipelines if not reconciled.

7.2 Future Work

Several directions for future work are identified.

The backend generator should be extended to support inter-entity relationships via `$ref` resolution, enabling the generation of JPA associations (e.g., `@OneToMany`, `@ManyToOne`) from schema composition. This would significantly increase the applicability of the platform to real-world domain models.

The frontend generated validation should be completed to derive runtime validation rules from schema constraints using a library such as `react-hook-form` combined with `Zod`, ensuring full parity between frontend and backend constraint enforcement.

Formal usability evaluation with representative technical and non-technical users should be conducted to assess the accessibility and efficiency of both authoring paths.

The LLM enhancement pipeline should be evaluated with fine-tuned model variants specifically trained on JSON Schema datasets to assess whether domain-specific training improves suggestion quality beyond the current general-purpose LLaMA-3.3-70B baseline.

Support for additional target frameworks including Angular, Vue, Django REST Framework, and Gradle-based Spring Boot should be implemented to broaden applicability.

The Redis caching layer should be fully implemented, and the quality scoring weights should be empirically calibrated against practitioner-defined criteria through a user study.

Chapter 8: Conclusion

This thesis presented FormSync, a schema-driven platform that addresses the fragmentation in form-based web application development by generating a consistent set of full-stack artefacts from a single JSON Schema source of truth. The platform was motivated by three well-documented problems in software engineering practice: cross-layer validation inconsistency, scaffolding overhead, and the difficulty of maintaining schema co-evolution across multiple layers of an application.

From a single schema definition, FormSync produces a WCAG 2.1-compliant React frontend application, a Spring Boot backend with complete CRUD infrastructure and OpenAPI documentation, and a constraint-driven JUnit 5 test suite eliminating the manual alignment required by all currently available approaches.

Five principal contributions were realised through this work. First, structural validation synchronisation across frontend, backend, and test layers was achieved through a shared JSON Schema source of truth and model-to-text generation pipelines. Second, constraint-driven boundary-value test data synthesis was implemented, deriving test inputs directly from schema metadata without requiring manual test authoring. Third, a provider-agnostic LLM enhancement pipeline with a three-layer validation safeguard and a quantitative quality scoring engine grounded in ISO/IEC 25010 was developed. Fourth, a multi-format plugin-based parser supporting JSON, YAML, and XML was implemented. Fifth, a persistent schema template repository was built, enabling reuse and versioning across development sessions.

Evaluation demonstrated that FormSync produces compilable, runnable artefacts across schemas of 5 to 40 fields with sub-two-second generation latency. The LLM enhancement pipeline achieved a 94.2% valid suggestion rate. Comparative analysis confirmed that FormSync outperformed JSONForms, JHipster, and OpenAPI Generator in end-to-end delivery time and cross-layer completeness.

The limitations of the current system including the stub frontend validator, the prototype WCAG accessibility checker, and the absence of inter-entity relationship support, are acknowledged and provide clear directions for future work. With these extensions, FormSync has the potential to become a comprehensive platform for schema-driven full-stack development applicable to a broader range of frameworks and real-world domain models.

References

- [1] A. Hunt and D. Thomas, *The Pragmatic Programmer*. Boston, MA, USA: Addison-Wesley, 1999.
- [2] F. Team, "Formik: Build forms in React, without tears," 2023. [Online]. Available: <https://formik.org/docs/overview>. [Accessed 14 Aug 2025].
- [3] K. Czarnecki and U. W. Eisenecker, *Generative Programming: Methods, Tools, and Applications*. Boston, MA, USA: Addison-Wesley, 2000.
- [4] R. H. F. Contributors, "React Hook Form Documentation," 2025. [Online]. Available: <https://react-hook-form.com>.
- [5] J. Team, "JSON Forms Documentation," JSONForms. [Online]. Available: <https://jsonforms.io/docs/>. [Accessed 12 Sep 2025].
- [6] S. Chen, L. Fan, T. Su, L. Ma, Y. Liu, and L. Xu, "Automated cross-platform GUI code generation for mobile apps," in *Proc. 1st IEEE Int. Workshop on Artificial Intelligence for Mobile*, Hangzhou, China, 2019.
- [7] S. Software, "OpenAPI Generator," OpenAPI Generator. [Online]. Available: <https://openapi-generator.tech>. [Accessed 10 Oct 2025].
- [8] V. Samotyy, U. Dzelendzyak, and N. Mashtaler, "A comparative study of data annotations and fluent validation in .NET," *International Journal of Computing*, vol. 23, no. 1, 2024, doi: 10.47839/ijc.23.1.3437.
- [9] J. Project, "JHipster - Full Stack Platform for the Modern Developer," JHipster. [Online]. Available: <https://www.jhipster.tech>. [Accessed 09 Oct 2025].
- [10] T. Beltramelli, "pix2code: Generating code from a graphical user interface screenshot," *arXiv preprint arXiv:1705.07962*, 2017.
- [11] S. Xiao, Y. Chen, J. Li, L. Sun, and T. Zhou, "Prototype2Code: End-to-end frontend code generation from UI design prototypes," in *Proc. ASME IDETC/CIE*, Washington, DC, USA, 2024.
- [12] Y. Duan, L. Yang, T. Zhang, Z. Song, and F. Shao, "Automated UI interface generation via diffusion models," in *Proc. Int. Symp. Computer Applications and Information Technology (ISCAIT)*, Xi'an, China, 2025.

- [13] D. C. Schmidt, "Model-driven engineering," IEEE Computer, vol. 39, no. 2, pp. 25–31, 2006.
- [14] M. Inayatullah, F. Azam, and M. W. Anwar, "Model-based scaffolding code generation for cross-platform applications," in Proc. IEEE 10th Annual IEMCON, Islamabad, Pakistan, 2019.
- [15] B. Zhang, Y. He, L. Pintscher, A. M. Penuela, and E. Simperl, "Schema generation for large knowledge graphs using large language models," arXiv preprint arXiv:2501.08608, 2025.
- [16] M. J. Mior, "Large language models for JSON schema discovery," arXiv preprint arXiv:2407.03286, 2024.
- [17] J. P. B. U. F. Neubauer, "AI-assisted JSON schema creation and mapping," arXiv preprint arXiv:2508.05192, 2024.
- [18] G. Inc., "Groq LPU Inference Engine and Performance Metrics," Groq Inc. [Online]. [Accessed 19 Sep 2025].
- [19] E. Zaidman and Contributors, "Ajv: Another JSON Schema Validator," 2024. [Online]. Available: <https://ajv.js.org/>. [Accessed 18 Sep 2025].
- [20] ISO/IEC, "Systems and software engineering - Systems and software quality requirements and evaluation (SQuaRE) - Product quality model," ISO/IEC 25010:2011, 2011.
- [21] W3C, "Accessible Rich Internet Applications (WAI-ARIA) 1.2," W3C. [Online]. Available: <https://www.w3.org/TR/wai-aria-1.2/>. [Accessed 13 Aug 2025].

Appendix A: Individual Contributions

The following is a summary of individual contribution areas inferred from the repository's commit history and code ownership. Precise line-level attribution is available from the git log.

- Yasas Rajapakshe: Primary ownership of the microservices architecture, API Gateway, Schema Enhancement Engine, User Management Service, and inter-service integration. Led the architectural design and the monorepo migration.
- Thihansi Gunawardena: Frontend and form-builder integration, routing-related frontend changes, and the frontend-form-builder service.
- Thamindu Dissanayake: Frontend UI layout and design, authentication UI integration, and related frontend enhancements.
- Movindu Liyanage: Microservice refactoring, deployment configuration, runtime-binding-engine integration, and inter-service communication.

Appendix B: API Endpoint Reference

Schema Enhancement Engine (port 3010 via gateway /schema)

- POST /schema/convert - Convert JSON/YAML/XML input to JSON Schema Draft-07.
- POST /schema/enhance - Run LLM-assisted enhancement and return quality metrics.
- POST /schema/suggestion/apply - Apply a selected suggestion to the current schema.
- POST /schema/quality/recalculate - Recalculate Q after suggestion application.
- GET/POST/PUT/DELETE /schema - CRUD operations on persisted schema records.

Frontend Form Builder Service (port 3014 via gateway /formgen)

- POST /generate-react - Generate and stream a React Vite project ZIP from a FormModel.

Backend DTO Generator (port 3012 via gateway /dto)

- POST /dto/generate - Generate and stream a Spring Boot project ZIP from a JSON Schema.
- POST /dto/preview - Return a JSON file listing without triggering a download.

Runtime Binding Engine (port 3013 via gateway /runtime)

- POST /runtime/generate - Generate a complete Spring Boot project with test suite.
- POST /runtime/preview - Return a JSON file listing for the generated project.

User Management Service (port 3011 via gateway /auth, /users, /template)

- POST /auth/register - Register a new user account.
- POST /auth/login - Authenticate and receive a JWT token.
- GET /users/profile - Retrieve the authenticated user's profile.
- GET/POST/DELETE /template - Manage saved schema templates.